

Module 4: Recursion on Linked Lists

1. Current progress analysis

You are ready for Module 4 because you already have the three required foundations.

From Module 1

You learned why linked lists exist.

Arrays are fast when you already know the index, but arrays have a fixed size after creation. Linked lists are more flexible because they can grow and shrink one node at a time.

Basic tradeoff:

Structure	Strength	Weakness
Array	Fast direct access by index	Fixed size, shifting needed for insertion/deletion
Linked list	Flexible size, easier pointer-based insertion/deletion	No direct jumping to an index

From Module 2

You learned what a node is.

A singly linked-list node has:

```
[data | next]
```

Meaning:

Part	Meaning
data	The value stored in the node
next	Address of the next node

You also learned the key pointer movement:

```
p = p->next;
```

That moves pointer `p` to the next node.

From Module 3

You learned traversal, display, and list creation.

Traversal means visiting nodes one by one:

```
first -> [8] -> [7] -> [12] -> NULL
```

Display with a loop:

```
void display(struct Node *p) {  
    while (p != NULL) {  
        printf("%d ", p->data);  
        p = p->next;  
    }  
}
```

Important rule from Module 3:

Do not move `first` during traversal. Use a temporary pointer like `p`.

Now Module 4 teaches the same traversal idea, but using recursion instead of a `while` loop.

2. Big idea of Module 4

Module 4 is about **recursion on linked lists**.

Recursion means:

A function calls itself to solve a smaller version of the same problem.

For linked lists, the smaller problem is easy to see.

Suppose the list is:

```
8 -> 3 -> 7 -> 12 -> 9 -> NULL
```

To display this list recursively, we can think like this:

```
Display 8, then display the rest: 3 -> 7 -> 12 -> 9 -> NULL
Display 3, then display the rest: 7 -> 12 -> 9 -> NULL
Display 7, then display the rest: 12 -> 9 -> NULL
Display 12, then display the rest: 9 -> NULL
Display 9, then display the rest: NULL
Stop.
```

So recursion naturally matches linked lists because each node points to the next smaller part of the list.

3. Recursive display compared with loop display

Iterative display from Module 3

```
void display(struct Node *p) {
    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
}
```

This says:

```
While p is not NULL:
    print current node
    move p to next node
```

Recursive display in Module 4

```
void RDisplay(struct Node *p) {
    if (p != NULL) {
        printf("%d ", p->data);
        RDisplay(p->next);
    }
}
```

This says:

```
If p is not NULL:  
    print current node  
    ask the same function to display the rest of the list
```

The loop version moves forward using:

```
p = p->next;
```

The recursive version moves forward by calling:

```
RDisplay(p->next);
```

Both visit the same nodes.

4. Base case

Every recursive function needs a stopping condition.

That stopping condition is called the **base case**.

For a normal singly linked list, the base case is:

```
p == NULL
```

Why?

Because `NULL` means there are no more nodes.

A safe recursive display can be written like this:

```
void RDisplay(struct Node *p) {  
    if (p == NULL) {  
        return;  
    }  
  
    printf("%d ", p->data);  
    RDisplay(p->next);  
}
```

Or like this:

```
void RDisplay(struct Node *p) {
    if (p != NULL) {
        printf("%d ", p->data);
        RDisplay(p->next);
    }
}
```

Both versions are correct.

The main safety rule is:

Check that `p` is not `NULL` before using `p->data` or `p->next`.

5. Forward recursive display

Forward display means printing the list in normal order.

Example list:

```
8 -> 3 -> 7 -> 12 -> 9 -> NULL
```

Expected output:

```
8 3 7 12 9
```

Code:

```
void RDisplay(struct Node *p) {
    if (p != NULL) {
        printf("%d ", p->data);
        RDisplay(p->next);
    }
}
```

The important order is:

```
printf("%d ", p->data);  
RDisplay(p->next);
```

Meaning:

Print first.
Then go to the next node.

6. Dry run: forward recursive display

List:

8 -> 3 -> 7 -> 12 -> 9 -> NULL

Call:

```
RDisplay(first);
```

Trace:

Call	p points to	Action
1	8	print 8, call RDisplay(3)
2	3	print 3, call RDisplay(7)
3	7	print 7, call RDisplay(12)
4	12	print 12, call RDisplay(9)
5	9	print 9, call RDisplay(NULL)
6	NULL	stop

Output:

8 3 7 12 9

This prints during the **calling phase**.

Calling phase means the time when function calls are still going deeper.

7. What is the system stack?

When a function is called, the computer stores information about that call.

That stored information is called an **activation record** or **stack frame**.

For recursion, each function call waits for the next function call to finish.

For this list:

```
8 -> 3 -> 7 -> 12 -> 9 -> NULL
```

The recursive calls are:

```
RDisplay(8)
RDisplay(3)
RDisplay(7)
RDisplay(12)
RDisplay(9)
RDisplay(NULL)
```

The stack can be imagined like this:

```
Top of stack

RDisplay(NULL)
RDisplay(9)
RDisplay(12)
RDisplay(7)
RDisplay(3)
RDisplay(8)

Bottom of stack
```

When the function reaches `NULL`, it stops going deeper.

Then calls begin to return one by one.

8. Calling phase and returning phase

Recursion has two important phases.

Calling phase

The function keeps calling itself.

Example:

```
RDisplay(8) calls RDisplay(3)
RDisplay(3) calls RDisplay(7)
RDisplay(7) calls RDisplay(12)
RDisplay(12) calls RDisplay(9)
RDisplay(9) calls RDisplay(NULL)
```

Returning phase

The function calls finish and return back to the previous calls.

Example:

```
RDisplay(NULL) returns to RDisplay(9)
RDisplay(9) returns to RDisplay(12)
RDisplay(12) returns to RDisplay(7)
RDisplay(7) returns to RDisplay(3)
RDisplay(3) returns to RDisplay(8)
RDisplay(8) finishes
```

This distinction matters because forward display prints during the calling phase, while reverse display prints during the returning phase.

9. Reverse recursive display

Reverse display means printing the list from last node to first node.

Example list:

```
8 -> 3 -> 7 -> 12 -> 9 -> NULL
```

Expected reverse output:

9 12 7 3 8

Code:

```
void RDisplayReverse(struct Node *p) {  
    if (p != NULL) {  
        RDisplayReverse(p->next);  
        printf("%d ", p->data);  
    }  
}
```

The important order is:

```
RDisplayReverse(p->next);  
printf("%d ", p->data);
```

Meaning:

First go all the way to the end.
Then print while returning.

This is the main trick of Module 4.

10. Dry run: reverse recursive display

List:

8 -> 3 -> 7 -> 12 -> 9 -> NULL

Call:

```
RDisplayReverse(first);
```

Calling phase

No printing happens yet.

```
RDisplayReverse(8)
RDisplayReverse(3)
RDisplayReverse(7)
RDisplayReverse(12)
RDisplayReverse(9)
RDisplayReverse(NULL)
```

At `NULL`, the function stops going deeper.

Returning phase

Now printing happens.

Returning from	Print
<code>RDisplayReverse(9)</code>	9
<code>RDisplayReverse(12)</code>	12
<code>RDisplayReverse(7)</code>	7
<code>RDisplayReverse(3)</code>	3
<code>RDisplayReverse(8)</code>	8

Output:

```
9 12 7 3 8
```

Important:

Reverse display does not reverse the actual linked list.

The real list is still:

```
8 -> 3 -> 7 -> 12 -> 9 -> NULL
```

Only the output order changed.

11. Forward vs reverse: the exact difference

Forward display:

```
printf("%d ", p->data);  
RDisplay(p->next);
```

Reverse display:

```
RDisplayReverse(p->next);  
printf("%d ", p->data);
```

Same two ideas:

```
print current node  
visit next node
```

But the order is different.

Version	First action	When printing happens	Output order
Forward recursive display	Print current node	Calling phase	First to last
Reverse recursive display	Call next node	Returning phase	Last to first

Changing the order of two lines changes the output order.

12. Full C program for Module 4

```
#include <stdio.h>  
#include <stdlib.h>  
  
struct Node {  
    int data;  
    struct Node *next;  
};  
  
struct Node *first = NULL;  
  
void create(int A[], int n) {  
    if (n <= 0) {  
        first = NULL;  
        return;  
    }  
  
    struct Node *t;
```

```

    struct Node *last;

    first = (struct Node *)malloc(sizeof(struct Node));
    first->data = A[0];
    first->next = NULL;
    last = first;

    for (int i = 1; i < n; i++) {
        t = (struct Node *)malloc(sizeof(struct Node));
        t->data = A[i];
        t->next = NULL;

        last->next = t;
        last = t;
    }
}

void RDisplay(struct Node *p) {
    if (p != NULL) {
        printf("%d ", p->data);
        RDisplay(p->next);
    }
}

void RDisplayReverse(struct Node *p) {
    if (p != NULL) {
        RDisplayReverse(p->next);
        printf("%d ", p->data);
    }
}

int main() {
    int A[] = {8, 3, 7, 12, 9};

    create(A, 5);

    printf("Forward display: ");
    RDisplay(first);

    printf("\nReverse display: ");
    RDisplayReverse(first);

    return 0;
}

```

Expected output:

Forward display: 8 3 7 12 9
Reverse display: 9 12 7 3 8

13. Instrumented version: showing recursion depth

This version prints messages so you can see the function going down and coming back.

```
void RDisplayTrace(struct Node *p) {  
    if (p == NULL) {  
        printf("Reached NULL. Start returning.\n");  
        return;  
    }  
  
    printf("Calling phase: visiting %d\n", p->data);  
    RDisplayTrace(p->next);  
    printf("Returning phase: back at %d\n", p->data);  
}
```

For this list:

8 -> 3 -> 7 -> NULL

The output will look like:

```
Calling phase: visiting 8  
Calling phase: visiting 3  
Calling phase: visiting 7  
Reached NULL. Start returning.  
Returning phase: back at 7  
Returning phase: back at 3  
Returning phase: back at 8
```

This makes the stack behavior easier to understand.

14. Number of recursive calls

If the list has `n` nodes, recursive display makes:

$n + 1$ calls

Why $n + 1$?

Because it calls once for every real node, plus one final call for `NULL` .

Example with 5 nodes:

8 -> 3 -> 7 -> 12 -> 9 -> NULL

Calls:

```
RDisplay(8)
RDisplay(3)
RDisplay(7)
RDisplay(12)
RDisplay(9)
RDisplay(NULL)
```

That is 6 calls.

So:

5 nodes + 1 NULL call = 6 calls

15. Complexity

For a list with n nodes:

Operation	Time complexity	Extra space	Why
Iterative display	$O(n)$	$O(1)$	Visits every node once using one pointer
Recursive forward display	$O(n)$	$O(n)$	Visits every node and stores recursive calls on the stack
Recursive reverse display	$O(n)$	$O(n)$	Visits every node and prints while returning

Why time is $O(n)$:

Every node is visited once.

Why recursive space is $O(n)$:

The system stack stores one activation record for each recursive call.

So recursion is elegant, but it uses more memory than a loop.

16. Common beginner mistakes

Mistake 1: Forgetting the base case

Wrong:

```
void RDisplay(struct Node *p) {  
    printf("%d ", p->data);  
    RDisplay(p->next);  
}
```

Problem:

When `p` becomes `NULL`, `p->data` is invalid.

Correct:

```
void RDisplay(struct Node *p) {  
    if (p != NULL) {  
        printf("%d ", p->data);  
        RDisplay(p->next);  
    }  
}
```

Mistake 2: Thinking reverse display reverses the list

Reverse display only prints in reverse order.

It does not change links.

This list:

```
8 -> 3 -> 7 -> NULL
```

After reverse display, the list is still:

```
8 -> 3 -> 7 -> NULL
```

Only the output was:

```
7 3 8
```

Mistake 3: Not understanding when printing happens

In forward display:

```
printf("%d ", p->data);  
RDisplay(p->next);
```

Printing happens before going deeper.

In reverse display:

```
RDisplayReverse(p->next);  
printf("%d ", p->data);
```

Printing happens after coming back.

Mistake 4: Thinking recursion is always better than loops

Recursion can make code clean, but it uses stack memory.

For very large lists, an iterative loop is usually safer and more memory efficient.

17. Mini practice

Practice 1

Given:

```
2 -> 4 -> 6 -> NULL
```

What does this print?

```
RDisplay(first);
```

Answer:

```
2 4 6
```

Practice 2

Given:

```
2 -> 4 -> 6 -> NULL
```

What does this print?

```
RDisplayReverse(first);
```

Answer:

```
6 4 2
```

Practice 3

How many recursive calls happen for this list?

```
10 -> 20 -> 30 -> NULL
```

Answer:

```
4 calls
```

Because:

```
RDisplay(10)  
RDisplay(20)  
RDisplay(30)  
RDisplay(NULL)
```

Practice 4

Why does printing after the recursive call reverse the output?

Answer:

Because the function first travels to the end of the list. It only prints while the calls return from the stack, so the last node prints first and the first node prints last.

18. Final Module 4 summary

Module 4 teaches recursive traversal.

The base case is:

```
p == NULL
```

Forward display:

```
printf("%d ", p->data);  
RDisplay(p->next);
```

Output:

```
first node to last node
```

Reverse display:

```
RDisplayReverse(p->next);  
printf("%d ", p->data);
```

Output:

```
last node to first node
```

The most important idea:

Printing before the recursive call gives normal order. Printing after the recursive call gives reverse order.

Complexity:

Time: $O(n)$
Space: $O(n)$ because of the system stack

Best next step:

Manually dry run this list until the calling phase and returning phase are clear:

8 -> 3 -> 7 -> NULL

Forward output:

8 3 7

Reverse output:

7 3 8